Studio Nx

Backend(FastAPI/Python) 40 — profile_image_file()

2026-10-12

Backend(FastAPI/Python) 40 — `profile_image_file()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-08 회원 정보: 사진 파일 찾기 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	39 `delete_profile_image()` · 41 `GET · PATCH /users/me`

1. 하는 일

주소에 들어 있는 사진 이름(`3f9c….jpg`)으로 **디스크에 있는 실제 파일 경로**를 찾아 돌려줍니다. 보여 주면 안 되는 경우는 모두 똑같이 404로 응답합니다.

`name`	결과
활동 회원의 사진 이름	`C:/…/studionx/storage/profiles/3f9c….jpg`
정지(`SUSPENDED`)된 회원의 사진	경로를 돌려줍니다 (보입니다)
탈퇴(`DELETED`)한 회원의 사진 (3일 안이어도)	404
아무도 쓰지 않는 이름	404
DB에는 있는데 디스크에 파일이 없음	404
`../../.env`, 대문자, `.gif`, `….jpg.exe`, 31자, 빈 문자열	**DB를 보지 않고** 404

404는 모두 `NOT_FOUND` "Image not found."입니다. 어느 경우인지는 알려 주지 않습니다.

2. 언제 쓰나

쓰는 곳	
`GET /api/v1/profile-images/{name}` (43)	화면의 `<img src>`가 부르며, 로그인 없이 동작합니다 (`<img>`에는 토큰을 붙일 수 없습니다). 이 함수가 돌려준 경로를 `FileResponse`로 보냅니다

35번의 사용자 결정에 따라, 회원 번호가 아니라 **사진을 올릴 때 만든 무작위 이름**으로 찾습니다.

3. 코드

import (파일 맨 위)

```
import re
from pathlib import Path, PurePosixPath
from uuid import uuid4

from fastapi import UploadFile
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session
```

```
from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
from app.models import User
from app.models.user import UserStatus
from app.schemas.user import MeResponse, MeUpdateRequest
from app.services.auth_service import check_nickname
```

`re`와 `UserStatus`를 이번에 추가했습니다.

상수 (이번에 추가)

```
# 주소로 받는 사진 이름 모양 — 올릴 때 만드는 uuid4().hex(소문자 16진수 32자) + 허용 확장자
PHOTO_NAME = re.compile(r"[0-9a-f]{32}\.(jpg|png|webp)")
```

함수

```
def profile_image_file(db: Session, name: str) -> Path:
    not_found = AppError(ErrorCode.NOT_FOUND, 404, "Image not found.")
    if not PHOTO_NAME.fullmatch(name):
        raise not_found
    user = db.scalar(select(User).where(User.profile_image_path == f"{PROFILE_DIR}/{name}"))
    if user is None or user.status == UserStatus.DELETED:
        raise not_found
    storage_dir = get_settings().storage_dir.resolve()
    path = (storage_dir / user.profile_image_path).resolve()
    if not path.is_relative_to(storage_dir) or not path.is_file():
        raise not_found
    return path
```

문법 풀이

import 문법	뜻
`import re`	정규식(글자 모양 검사)을 씁니다
`from app.models.user import UserStatus`	회원 상태 목록입니다 (P1-03 08)

식	뜻
`re.compile(r"…")`	정규식을 미리 한 번 만들어 둡니다. `r"…"`은 `\`를 그대로 쓰는 문자열입니다
`[0-9a-f]{32}`, `\.`, `(jpg|png|webp)`	차례로 소문자 16진수 32개, 점, 셋 중 하나를 뜻합니다
`PHOTO_NAME.fullmatch(name)`	**처음부터 끝까지** 모양이 맞아야 참입니다 (`….jpg.exe`는 거짓)
`.resolve()`	`..`과 바로 가기(심볼릭 링크)를 풀어서 진짜 절대 경로로 바꿉니다
`path.is_relative_to(storage_dir)`	경로가 그 폴더 **안에** 있는지 확인합니다 (Python 3.9 이상)
`path.is_file()`	디스크에 파일로 존재하는지 확인합니다

4. 흐름

1. 이름의 모양이 맞지 않으면 404로 응답합니다 (DB를 보지 않습니다).
2. 그 이름을 쓰는 회원을 찾습니다. 없거나 탈퇴한 회원이면 404로 응답합니다.
3. 실제 경로를 만들고, storage 폴더 안에 있는지와 파일이 있는지 확인합니다. 아니면 404로 응답합니다.
4. 경로를 돌려줍니다.

5. 왜 이렇게

선택	이유
이름 모양 검사를 **맨 앞**에 둠	`../../.env` 같은 이름은 DB를 보기도 전에 끝납니다. 모양이 맞는 이름만 경로가 될 수 있으므로, 폴더 밖으로 나가는 길이 처음부터 막힙니다
그래도 `is_relative_to`를 한 번 더 확인	템플릿이 요구한 사항입니다. DB 값이 어떤 이유로든(손으로 고치는 등) 이상해지더라도 storage 밖의 파일을 보내지 않습니다. 이중 잠금인 셈입니다
탈퇴 회원은 3일 안이어도 404	탈퇴는 "사이트에서 나를 지운다"는 뜻입니다. 파일 자체는 P1-09 탈퇴가 39번으로 지우지만, 그 전에도 보이지 않게 합니다. 추적 방지 ①과도 맞습니다
정지된 회원의 사진은 보임	정지는 관리자의 조치일 뿐이라서, 협업 화면(Phase 6)의 다른 멤버에게까지 사진을 숨길 이유가 없습니다. 바꾸려면 조건 한 줄만 고치면 됩니다
모든 404를 같은 문구로 응답	어떤 이름이 "있었는데 탈퇴했는지" 아니면 "처음부터 없었는지" 구분할 수 없게 합니다
소문자만 허용	`uuid4().hex`는 항상 소문자입니다. DB 비교는 대소문자를 무시하지만, 주소는 한 가지 모양만 받아서 같은 사진이 여러 주소로 열리지 않게 합니다
대문자나 형식 오류도 404 (422가 아님)	파일 주소이므로 "그런 파일이 없다"는 응답이 자연스럽습니다

알아 둘 점 (색인): `profile_image_path` 칸에는 색인(index)이 없어서, 사진을 열 때마다 회원 표를 처음부터 훑습니다. 회원이 수천 명 정도까지는 문제가 없습니다. 회원이 많아지면 `users.profile_image_path`에 UNIQUE 색인을 추가하면 됩니다 (DB.md 변경이 필요하므로 그때 사용자와 정합니다).

6. 시험 (`tests/test_user_service.py`에 12개를 추가해서 모두 56개, 실제 MySQL과 임시 폴더로 실행)

시험	확인한 것
활동 회원의 사진	storage 안의 절대 경로를 돌려주고, 파일 내용이 같습니다
이상한 이름 7가지	`../../.env`, 인코딩된 `..%2F`, 대문자, `.gif`, `.jpg.exe`, 31자, 빈 문자열은 모두 404입니다
아무도 쓰지 않는 이름	404
탈퇴한 회원	404
정지된 회원	경로를 돌려줍니다
디스크에 파일이 없음	404

모든 404가 `NOT_FOUND`와 "Image not found."로 같다는 것을 확인했습니다. 전체 시험 290개가 통과했고, `ruff` 검사도 통과했습니다.